第36课 DFS经典题 —— 迷宫探路与四皇后

一、课程信息

项目	内容
课次	第 36 课 / 共 40 课
主题	DFS 经典题（迷宫问题、八皇后简化版）
前置知识	第35课：DFS入门（全排列、回溯）
后续衔接	第37课：BFS入门

教学目标（可检验）

1. 学生能独立用 DFS + 方向数组完成迷宫路径搜索，正确处理边界判断与访问标记。
2. 学生能用逐行放置 + 回溯的方法，写出 4 皇后问题的完整代码并输出所有合法方案。
3. 学生能总结 DFS 解题的通用模板，并说出"标记递归回溯"三步口诀。

二、教学重点与难点

重点

- 方向数组 `dx[]`、`dy[]` 的使用方法与含义
- 迷宫中边界判断 + 墙壁判断 + 访问标记的"三重检查"
- 4 皇后问题中列冲突、对角线冲突的检查逻辑
- DFS 万能模板的提炼与记忆

难点

- 二维坐标 `(x, y)` 与数组下标的对应关系（孩子容易搞混行列）
- 对角线冲突判定公式 `|row1 - row2| == |col1 - col2|` 的理解
- 回溯时"擦掉脚印"（取消标记）的时机与原因

三、课堂时间分配（共 100 分钟）

时间段	时长	内容	形式
0′—5′	5 min	复习热身：回顾第35课 DFS + 回溯口诀	师生问答
5′—25′	20 min	新课讲解一：DFS 走迷宫（故事引入 + 画图演示）	讲授 + 板书画图
25′—45′	20 min	例题1实战：迷宫方案数（学生跟写 + 运行）	讲练结合
45′—55′	10 min	课堂练习一：迷宫变式题（独立完成）	学生练习
55′—60′	5 min	休息 + 自由提问	休息
60′—80′	20 min	新课讲解二：4皇后问题（棋盘故事 + 画图推演）	讲授 + 画图
80′—90′	10 min	例题2实战：4皇后代码（学生跟写 + 运行）	讲练结合
90′—95′	5 min	DFS 万能模板总结 + 对比全排列/迷宫/皇后	归纳总结
95′—100′	5 min	课堂练习二 + 布置作业 + 下节课预告	收尾

> 说明：两段讲授各 20 分钟，均不超过 25 分钟要求。每段讲授后紧跟动手实践。

四、讲稿提纲

第一部分：DFS 走迷宫（5′—25′）

1. 故事引入（2 分钟）

> 同学们，想象一下——你被关进了一座大迷宫里！四面都是高高的墙壁，你只能往上、下、左、右四个方向走。你身上带了一桶红油漆，每走过一个格子就在地上画个红叉，这样就不会重复走了。如果走到死胡同怎么办？退回来，换条路试试！这就是我们上节课学的——**回溯**。
>
> 今天我们就来教计算机走迷宫！

2. 迷宫的表示方法（3 分钟）

- **用二维数组当地图**：`maze[5][5]`，0 表示路（可以走），1 表示墙（不能走）。
- 在黑板上画一个 5×5 的格子，涂黑几个格子当墙壁，标出起点 S 和终点 E。
- 类比：二维数组就像教室里的座位表，第几排第几列就能找到一个位置。

```
地图示例（5×5）：
 0 1 2 3 4
0 [S][0][1][0][0]
1 [0][0][1][0][1]
2 [1][0][0][0][1]
3 [0][1][1][0][0]
4 [0][0][0][1][E]
```

3. 方向数组——教计算机"认路"（5 分钟）

> 我们人类知道上下左右，计算机怎么知道？我们给它一张"小纸条"：

```
int dx[] = {0, 0, 1, -1}; // 行的变化：右左不动，下+1，上-1
int dy[] = {1, -1, 0, 0}; // 列的变化：右+1，左-1，上下不动
```

- 画图演示：站在 `(2,1)` 这个格子，分别用 4 个方向计算出新坐标。
- 口诀："**方向数组成对记，dx 管行 dy 管列**"。

4. 三重检查——安全第一（5 分钟）

> 在迷宫里走路之前，要检查三件事：

1. **有没有越界？** 新位置的行和列都要在 0 到 n-1 之间（别走出迷宫外面了！）
2. **是不是墙壁？** `maze[nx][ny] == 1` 就是墙，撞墙可不行。
3. **走过没有？** `vis[nx][ny] == 1` 表示来过了（红油漆标记），不能重复走。

> 三个检查全通过，才能迈出这一步！

5. DFS 走迷宫流程（5 分钟）

在黑板上用箭头模拟 DFS 走迷宫的完整过程（走到底回溯换方向），重点演示：
- 走到死路时怎样"退回来"（回溯）
- 退回来时要"擦掉红油漆"（取消 vis 标记）
- 找到终点时怎么处理（方案数 +1，或者打印路径）

第二部分：4 皇后问题（60′—80′）

1. 故事引入（3 分钟）

> 同学们玩过国际象棋吗？里面有一种最厉害的棋子叫"皇后"（Queen），她能攻击同一行、同一列、同一斜线上的所有棋子。
>
> 现在问题来了：在一个 4×4 的小棋盘上，放 4 个皇后，让她们谁也打不到谁，有几种放法？
>
> 这就是大名鼎鼎的"八皇后问题"的简化版——我们先从 4 个皇后开始！

2. 分析思路（7 分钟）

- **每行只能放一个皇后**（因为同行会互相攻击），所以我们一行一行放。
- 第 0 行放哪一列？第 1 行放哪一列？……这就是"逐行搜索"。
- **冲突检查**：
- 同列冲突：两个皇后列号相同 用一个数组 `col[]` 记录。
- 对角线冲突：`|行差| == |列差|` 画图展示两个对角线方向。
- 画出 4×4 棋盘，手动演示放皇后的过程：放检查冲突就换位置放满4个就记录方案。

3. 回溯过程演示（5 分钟）

- 在黑板上画搜索树：第0行4种选择 第1行若干合法选择 ……
- 强调：搜索树上"走不通就退回来"就是回溯！
- 最终 4 皇后一共有 **2 种** 合法方案，让学生先猜，再验证。

4. 代码实现要点（5 分钟）

- `queen[i]` 记录第 i 行皇后放在第几列
- `check(row, col)` 函数：检查在 `(row, col)` 放皇后是否与前面已放的皇后冲突
- DFS 函数：逐行尝试每一列，合法就放，放满 n 行就输出方案

第三部分：DFS 万能模板总结（90′—95′）

> 同学们，我们学了全排列、迷宫、皇后，它们的 DFS 长得都很像！记住这个万能三步：

口诀："标记—递归—回溯"

```
void dfs(当前状态) {
 if (到达目标) {
 记录/输出答案;
 return;
 }
 for (每一种选择) {
 if (选择合法) {
 做标记;
 dfs(下一个状态);
 取消标记; // 回溯!
 }
 }
}
```

对比三道题：

	全排列	迷宫	4皇后
当前状态	已选了几个数	当前站在哪个格子	已放了几行
选择	选哪个数字	往哪个方向走	这一行放哪一列
标记	used[i]=true	vis[x][y]=true	queen[row]=col
回溯	used[i]=false	vis[x][y]=false	（自动覆盖）

七、常见错误与应对

错误 1：忘记标记起点

```
// 错误写法：直接调用 dfs(0, 0)，没有先标记起点
dfs(0, 0);

// 正确写法：
vis[0][0] = 1; // 起点也要标记！
dfs(0, 0);
```

> **应对**：提醒学生——"你自己站的那个格子，也要画上红油漆，不然你可能绕一圈又走回起点。"

错误 2：忘记回溯（不擦红油漆）

```
// 错误：少了 vis[nx][ny] = 0
vis[nx][ny] = 1;
dfs(nx, ny);
// 这里没有 vis[nx][ny] = 0; 少了这行！
```

> **结果**：只能找到一条路（或更少），因为其他路径被"红油漆"挡住了。
>
> **应对**：强调"标记和取消标记永远成对出现"，写代码时先把框架写好再填内容。

错误 3：方向数组写错 / dx 和 dy 搞反

```
// 错误示例：dx 和 dy 的值写反了
int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};
// 这个其实没错（上下左右的顺序不同而已），但如果写成：
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 1, -1}; // 这就错了！变成了对角线移动
```

> **应对**：让学生在纸上画 `(x,y)` 的上下左右坐标，亲手验证方向数组是否正确。

错误 4：对角线冲突判断少写一半

```
// 错误：只检查了一个方向的对角线
if (i - row == queen[i] - col) // 只查了"\"方向

// 正确：用绝对值同时检查"/"和"\"两个方向
if (abs(i - row) == abs(queen[i] - col))
```

> **应对**：在棋盘上画出两条对角线，让学生数格子验证。

错误 5：数组下标从 0 还是从 1 开始搞混

> 迷宫题我们下标从 0 开始，皇后题列号从 1 开始——两道题的习惯不一样。
> **应对**：每道题开始前，先跟学生约定好"下标从 0 还是 1 开始"，写在注释里。

九、下节课预告

> 同学们，今天我们学会了用 DFS 走迷宫和放皇后。DFS 的特点是"一条路走到黑，走不通就回头"。
> 下节课我们学另一种搜索方法——**BFS（广度优先搜索）**。
> 如果说 DFS 像一个人拿着绳子在迷宫里一路走到底，那 BFS 就像在水面上丢一块石头——**水波一圈一圈向外扩散**，先到的地方一定距离最近！BFS 天生就能找到最短路径，是解迷宫最短路问题的利器。
> 请同学们预习：队列（queue）的用法，复习第31课栈与队列的内容。下节课见！